Smart Manufacturing Multi-Agent System with a Reduced SLM Footprint

Darsh Veer Singh (IMT2023543), Rakshith Srinivasan (IMT2023544)
International Institute of Information Technology, Bangalore
DarshVeer.Singh@iiitb.ac.in, *Rakshith.Srinivasan@iiitb.ac.in*

***Abstract*—This report presents a practical smart-manufacturing multi-agent system for predictive maintenance, anomaly detection, and prescriptive decision support. The implementation combines deterministic machine-learning pipelines with selective use of cloud and edge language models. Compared with the earlier conceptual architecture, the repository now follows a rules-first execution style: schema discovery, preprocessing, model-family selection, and most orchestration choices are performed deterministically, while language models are retained only where they still add value.**

The system supports CSV and NPZ dataset ingestion, auto-detection of task type, supervised and anomaly-detection workflows, pretrained inference, model caching, adaptive retry, synthetic-data generation, and a local web interface for execution monitoring. The final design reduces local SLM usage from four proposed positions to one retained anomaly-parameter suggestion step, while the summary stage uses a cloud Reflexion loop or a fallback summary path. This makes the system more reproducible, easier to debug, and closer to deployment-ready smart-manufacturing software.

***Index Terms*—Smart manufacturing, multi-agent systems, agentic AI, prescriptive maintenance, large language models, small language models, anomaly detection.**

I. Introduction

Modern smart-manufacturing environments generate large volumes of sensor, operational, and maintenance data through Industrial Internet of Things (IIoT) devices and cyber-physical production systems. Traditional predictive-maintenance pipelines can detect degradation and estimate failures, but industrial workflows also require prescriptive reasoning, explainability, workflow traceability, and deployment-friendly execution.

This project implements a modular smart-manufacturing multi-agent system that combines deterministic machine-learning pipelines with selective Large Language Model (LLM) orchestration. The repository was originally described as a broader multi-SLM architecture, but the implemented system now uses a narrower and more auditable control flow. The final design keeps language-model use where it is still useful for reasoning over ambiguous or narrative tasks, but avoids unnecessary SLM calls where thresholds and schema statistics are sufficient.

The system supports:

- dataset inspection and schema discovery,
- preprocessing and feature analysis,
- classification, regression, and anomaly detection,
- adaptive retry for weak-performing models,
- recommendation generation,
- Reflexion-based workflow summarization,
- pretrained inference and model caching,
- synthetic-data generation and evaluation,
- and frontend-assisted execution through a local web application.

The implementation originally explored multiple local Small Language Model (SLM) roles for preprocessing, perception, anomaly detection, and summarization. After comparing the repository documentation, runtime code, and supporting notes, most of those local SLM calls were replaced by deterministic logic because the decisions were enumerable from dataset statistics and implementation rules. The final implementation therefore retains one local SLM role for anomaly-parameter suggestion, while the remaining stages rely on rule-based logic or cloud-based summarization.

Relative to the original paper concept, the important shift is not only architectural but operational. The repository now exposes a repeatable workflow with visible checkpoints, cached artifacts, pretrained bundles, and a web dashboard. That makes it possible to evaluate the system as a concrete machine-learning platform rather than only as an orchestration idea.

A. Contribution summary

The implementation makes four practical contributions beyond the initial conceptual design:

- a rules-first orchestration path that removes repeated language-model decisions from the core pipeline,
- pretrained inference and hash-keyed caching for repeatable runs,
- anomaly-detection support with a retained local SLM only where parameter selection is genuinely ambiguous,
- and a local web application that exposes each workflow stage and persists outputs as artifacts.

II. Methodology

A. Implementation map

Table I links the main runtime stages to the concrete modules that implement them.

B. Three-tier intelligence hierarchy

The final implementation follows a reduced three-tier architecture, as illustrated in Fig. 1:

TABLE I
STAGE-BY-STAGE IMPLEMENTATION MAP

Stage	Modules	What happens	Output
0	main_llm.py, auto_detect.py, pretrained_model_store.py	Load CSV or NPZ data, infer or confirm the task type, infer the target column when needed, and match the dataset to a pretrained bundle if inference-only mode is enabled. HITL is used when detection confidence is low.	Resolved task configuration
1	data_loader_agent.py, schema_discovery.py, column_utils.py	Profile the dataset shape, dtypes, missing values, identifier-like columns, and target-signal diagnostics.	Dataset profile and schema report
2	preprocessing_agent.py, tool_decider.py, intelligent_feature_analysis.py	Select imputation, scaling, and encoding rules; drop leakage-prone or high-cardinality columns; optionally apply PCA; preserve protected identifiers for anomaly workflows.	Processed feature matrix
3	dynamic_analysis_agent.py, model_cache.py, pretrained_model_store.py, prediction_analyzer.py	Run pretrained inference or live training, check the cache, perform adaptive retry when metrics are weak, and compute evaluation metrics.	Predictions and model metrics
4	optimization_agent.py	Convert predictions into maintenance priorities, anomaly explanations, confidence estimates, and recommended actions.	Recommendation table
5	intelligent_summarization.py, llm_output_logger.py, reporting.py	Generate the workflow summary with a cloud Reflexion loop or fallback summary path and store structured logs.	Narrative summary and audit trace
6	webapp/app.py, run_manager.py, run_local_app.py	Expose the workflow in a local FastAPI UI, persist artifacts, and support synthetic generation and download.	Browser-based run dashboard

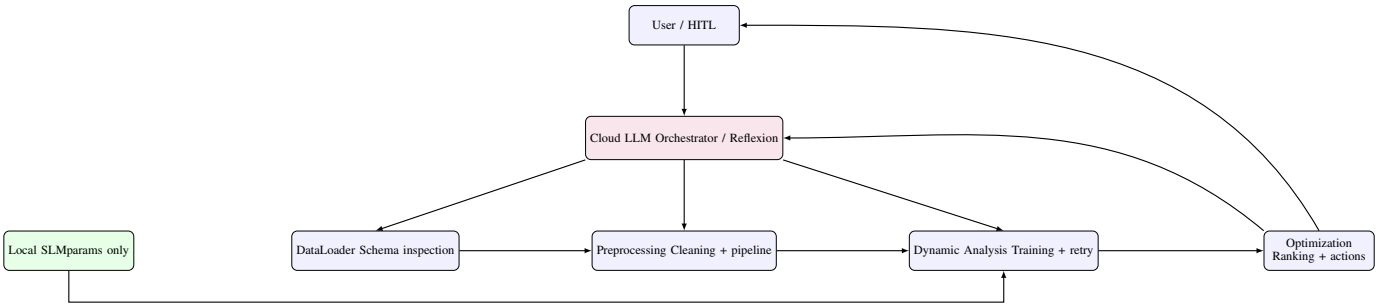

Fig. 1. Reduced smart-manufacturing architecture. The current implementation keeps one orchestrator LLM and one retained SLM call for anomaly-parameter suggestion; all other local SLM roles are replaced by deterministic logic.

- **Tier 1 - Cloud LLM layer:** Handles orchestration in the older LLM mode and performs the final Reflexion-based workflow summary in rules-first mode.
- **Tier 2 - Local SLM layer:** Retained only for anomaly-parameter suggestion during Isolation Forest configuration.
- **Tier 3 - Rule-based layer:** Handles preprocessing decisions, schema inspection, model-family selection, and cache/inference routing using deterministic thresholds.

This architecture reduces hallucination risk, improves reproducibility, lowers latency, and simplifies debugging while preserving flexibility for anomaly-detection tasks. The code base shows that most fully enumerable decisions are not model problems at all, so removing those SLM calls is a quality improvement rather than a simplification by omission.

C. Execution modes

The repository provides two orchestration modes. In the original `llm` mode, the planner chooses the next agent at each step and the final summary is generated afterward. In the newer `rules-first` mode, the deterministic pipeline runs first and the cloud model is called only once at the end.

This distinction matters because it changes the unit of uncertainty. In `llm` mode, every step is a potential language-model decision. In `rules-first` mode, the only generative step is the final summary, while the important data-path decisions are handled by code and thresholds.

D. Workflow pipeline

The final implementation follows the workflow shown below:

1) Dataset upload or built-in dataset selection
2) Schema discovery and metadata extraction
3) Preprocessing and feature analysis
4) Optional PCA compression
5) Model training or pretrained inference
6) Adaptive retry for weak-performing models
7) Recommendation generation and optimization
8) Artifact generation and logging
9) Reflexion-based workflow summarization
10) Frontend visualization and export

Unlike the earlier conceptual workflow, the current repository executes most stages deterministically before the LLM is invoked. This rules-first approach improves stability and reproducibility, especially when the same dataset is processed multiple times with the cache or pretrained bundles enabled.

E. Stage 0: data loading and task resolution

The workflow begins by loading the dataset through a unified loader that accepts CSV and NPZ inputs. This is important because the repository does not assume one narrow file format. The loader first inspects the file extension and then either reads tabular CSV data directly or reconstructs a DataFrame from NPZ arrays when the data is stored in bundled form.

After loading, the pipeline resolves the target column and problem type. If a target is not provided, the system can infer it from the dataset, or from pretrained bundle compatibility when inference-only mode is enabled. If the inferred problem type is uncertain, a HITL confirmation gate is used when the confidence is below the configured threshold.

This step is crucial because it determines the downstream path. Classification and regression require a valid target column, while anomaly detection explicitly runs without one.

F. Stage 1: schema discovery and diagnostics

The schema-discovery stage records the operational shape of the dataset before any transformation is applied. The implementation logs the number of rows and columns, column dtypes, missing-value counts, and sample statistics. It also identifies identifier-like columns and target-signal diagnostics so that later stages can avoid accidental leakage or confusing column names.

The documentation and code agree that identifier detection must be explicit rather than casual heuristic matching. Identifier columns such as machine IDs, asset IDs, or agent IDs are important for post-prediction recommendations, but they should not be treated as normal predictive inputs. The implementation therefore distinguishes between columns that are informative for traceability and columns that should be excluded from training.

G. Stage 2: preprocessing and feature engineering

The preprocessing pipeline is rule-based. The ToolDecider examines dataset size, missing-value percentage, numeric/categorical composition, and memory footprint to choose the preprocessing recipe. The implementation then applies imputation, scaling, and encoding, with the following behavior:

- missing values above a threshold can trigger KNNImputer, otherwise SimpleImputer is used,
- larger datasets favor RobustScaler, while smaller datasets use StandardScaler,
- normalization can be selected when a MinMax-style transformation is appropriate,
- low-cardinality categoricals are one-hot encoded,
- high-cardinality categoricals are dropped to avoid exploding feature spaces,
- target columns are removed to prevent leakage,
- optional PCA can compress the output when interpretability is less important than dimensionality reduction.

The preprocessing stage is also where feature analysis occurs. For supervised tasks, the implementation computes target relationships and recommends removing columns that add little value or create leakage risk. For anomaly detection, protected identifier columns can be carried forward so that anomalies can still be mapped back to real machines or assets after prediction.

The key methodological point is that preprocessing choices are not guessed by an SLM. They are derived from the data profile and from explicit rules, which makes the same dataset yield the same preprocessing path across repeated runs.

H. Stage 3: model analysis, pretrained inference, and adaptive retry

The analysis stage performs the central machine-learning task. It supports three problem families: classification, regression, and anomaly detection.

For classification, the runtime can use Logistic Regression, Random Forest, or SVC. For regression, it can use Linear Regression, Ridge, Lasso, HistGradientBoostingRegressor, Random Forest Regressor, or SVR. For anomaly detection, Isolation Forest is used as the dedicated detector.

There are three important implementation details here.

First, the system can run in pretrained inference mode for supervised tasks. In that path, the model bundle is loaded from the pretrained registry and applied directly to the new dataset. This is useful when the target dataset has already been trained offline and the goal is to evaluate or deploy the model rather than retrain it.

Second, the system uses hash-keyed caching. The cache key depends on the dataset basename, sorted feature list, target column, and problem type. If the same configuration is seen again, the trained model is reused immediately.

Third, the system includes adaptive retry. If the initial classification accuracy falls below 0.60 or the regression R^2 falls below 0.10, the runtime sweeps the remaining model

families and picks the best result. This is a practical safeguard for cases where the first model family is a poor fit for the data geometry.

The anomaly path is intentionally different. The retained SLM is allowed to suggest contamination and estimator settings because those values depend on the distribution shape and false-positive trade-offs. The SLM output is bounded before use, and the code can fall back to safe defaults if the SLM response is missing or malformed.

I. Stage 4: recommendation generation

The optimization stage converts predictions into maintenance actions. In classification mode, predicted labels are mapped into priority scores and then into action plans with estimated cost and time frame. In regression mode, the system identifies predictions that exceed warning or critical thresholds derived from the training distribution. In anomaly-detection mode, it groups anomalous records by machine or asset, aggregates anomaly scores, and extracts the strongest contributing signals from per-feature z-scores.

This stage is important because it closes the loop from prediction to action. The output is not just a score or a label. It is a maintenance-oriented table that explains what should happen next, why it should happen, and how urgent it is.

J. Stage 5: summarization, logging, and traceability

The final narrative summary is generated by the intelligent summarization module. The preferred path uses a cloud Reflexion loop with draft, critique, and revision passes. If the cloud model is unavailable, the runtime can fall back to a local summary path or, as a last resort, a rule-based text summary.

The important methodological point is that this stage is now only about interpretation. It does not control the core workflow. That separation keeps the summary layer useful without allowing it to compromise the actual data pipeline.

Structured logging also matters. The implementation records run summaries, recommendations, and artifact locations so that a later reviewer can reconstruct what happened in a particular run. This is especially useful when comparing live-training runs with pretrained inference runs or when evaluating cache hits versus cache misses.

K. Stage 6: web application and synthetic-data workflow

The repository also includes a local FastAPI application and a background run manager. The frontend allows a user to upload a dataset, select a built-in dataset, start a run, and inspect stage-by-stage progress. The run manager captures logs, stores stage summaries, and writes run artifacts to the web-specific artifact directories.

Synthetic-data generation is integrated into the web workflow and the standalone scripts. The generator samples numeric and categorical columns from the observed distributions, optionally preserves target structure, and then evaluates the synthetic dataset using a quality analyzer and a prediction analyzer. This gives the repository a useful validation path beyond ordinary train-test evaluation.

TABLE II
REPOSITORY RUNTIME STORAGE STRUCTURE

Directory	Purpose
`logs/`	Workflow logs, summaries, and audit traces
`model_cache/`	Cached trained models keyed by dataset and feature set
`artifacts/web_runs/`	Web UI run outputs and summaries
`artifacts/web_uploads/`	Uploaded datasets
`artifacts/web_synthetic/`	Synthetic datasets and inference reports
`artifacts/pretrained_models/`	Exported pretrained bundles and registry metadata

L. Runtime storage and artifacts

Table II summarizes the repository directories used at runtime.

M. Auto-detection and HITL policy

The auto-detection policy is conservative. When the task type is obvious from the target column or the data shape, the system proceeds automatically. When the confidence is low, a HITL confirmation gate is shown unless the user has explicitly enabled auto-approval. This is a useful compromise between autonomy and safety in a manufacturing setting, where the wrong task type can produce misleading recommendations.

In practice, this means the system prefers certainty at the task-selection step and speed later in the pipeline. Once the configuration is resolved, the remainder of the workflow can run unattended if needed.

N. How this differs from the earlier concept

The core contrast with the earlier paper concept is now easy to state. The original design leaned on several local SLM roles to reason about schema discovery, preprocessing, model choice, anomaly settings, and summary generation. The implemented repository keeps only one local SLM role, and only because anomaly parameter selection still contains enough ambiguity that a bounded natural-language suggestion can be helpful.

That reduction is not a loss of capability in the other stages. It is a shift toward deterministic engineering where the decision space is fully enumerable. The result is a smaller surface for failures and a clearer separation between data logic, model logic, and narrative logic.

III. RESULTS AND DISCUSSION

The implemented system was evaluated using multiple manufacturing datasets covering classification, regression, and anomaly-detection scenarios.

A. Classification workflow

On the Smart Manufacturing Maintenance Dataset, Random Forest classification produced the strongest performance in the reported runs, with an accuracy of 97.4%. The preprocessing pipeline successfully identified informative sensor features and generated ranked maintenance recommendations.

The main practical benefit of the rules-first design is that the classification result is easier to reproduce. Because the preprocessing recipe and model-family selection are deterministic, repeated runs on the same dataset are much less sensitive to prompt variation or planner drift.

B. Regression workflow

Regression experiments demonstrated the usefulness of adaptive retry logic. When the initial regression model produced weak performance, the system automatically evaluated alternative model families such as Ridge Regression, Lasso Regression, Random Forest Regressor, HistGradientBoostingRegressor, and SVR. The best-performing model achieved an R^2 of 0.9925 on the Smart Manufacturing Maintenance Dataset.

This adaptive behavior matters because manufacturing regression targets are often noisy or weakly correlated with the available sensor features. In those cases, a single default model can fail silently or underperform badly. The retry path makes that failure visible and gives the user a better fallback model without manual experimentation.

C. Anomaly-detection workflow

The anomaly-detection workflow used Isolation Forest on a large manufacturing dataset. The retained local SLM suggested contamination and estimator values before the detector executed, and the output was then converted into grouped maintenance actions.

The anomaly detector system flagged approximately 23.3% of the samples as anomalies, with vibration, latency, and packet-loss features contributing most strongly to the detections on the Smart Manufacturing Maintenance Dataset. This retained SLM role is justified because contamination selection depends on the distribution shape, the expected outlier rate, and the cost of false alarms. Those are not simple thresholding problems.

D. Pretrained inference and cache behavior

The pretrained inference path is one of the clearest signs that the repository has moved beyond a conceptual prototype. Instead of retraining every time, the workflow can load a saved bundle, align the expected feature columns, and score the new dataset directly. This makes repeated experiments significantly faster and also gives the report a deployment-oriented dimension.

The cache path serves a different purpose. It avoids recomputing the same supervised model when the dataset basename, feature set, target, and problem type are unchanged. In practice, this is especially useful for local experimentation because repeated runs often change only a small part of the configuration.

E. Synthetic inference evaluation

Synthetic-data evaluation demonstrated that the pretrained models generalized well to unseen generated samples.

Key observations included:

TABLE III
SLM REDUCTION STRATEGY

Slot	Task	Keep	Replacement
SLM 1	Schema discovery	No	Pandas + rules
SLM 2	Preprocessing strategy	No	ToolDecider rules
SLM 3a	Model selection	No	Deterministic lookup
SLM 3b	Anomaly parameters	Yes	Retained local SLM
SLM 4	Workflow summary	No	Reflexion cloud summary

- synthetic regression predictions remained close to training distributions,
- classification distributions matched the original datasets,
- synthetic quality metrics compared the source and generated data directly,
- and the prediction analyzer provided an additional summary of the generated outputs.

For regression inference, the synthetic prediction mean differed from the training mean by approximately 2.43 points in the reported runs, which suggests that the pretrained model still tracked the general scale of the target after synthetic perturbation.

The synthetic-data workflow therefore provides an extra validation layer beyond train-test evaluation. It is especially useful when the goal is to inspect whether the model behaves plausibly on data with similar distributions rather than only on a single holdout split.

Figure 2 shows the frontend outputs for predictions on synthetic data generated from the Smart Maintenance Dataset (500 rows).

F. Frontend-assisted execution

The local frontend significantly improved usability and workflow visualization. Instead of relying only on CLI execution, users could upload datasets, observe workflow stages, inspect recommendations, and download generated artifacts directly from the browser.

This front-end layer also makes the repository easier to demonstrate in a classroom or lab setting because each stage is visible as a run artifact rather than hidden behind terminal output.

G. SLM reduction summary

One of the major outcomes of the project was the reduction of local SLM usage from four positions to one retained role.

The reduction simplified the architecture while preserving the one local reasoning capability that still benefits from flexible judgment.

H. Discussion of robustness

The strongest practical advantage of the current implementation is not raw model accuracy; it is operational clarity. Each stage can be inspected independently, each output can be saved or reloaded, and each generative step is isolated. That makes

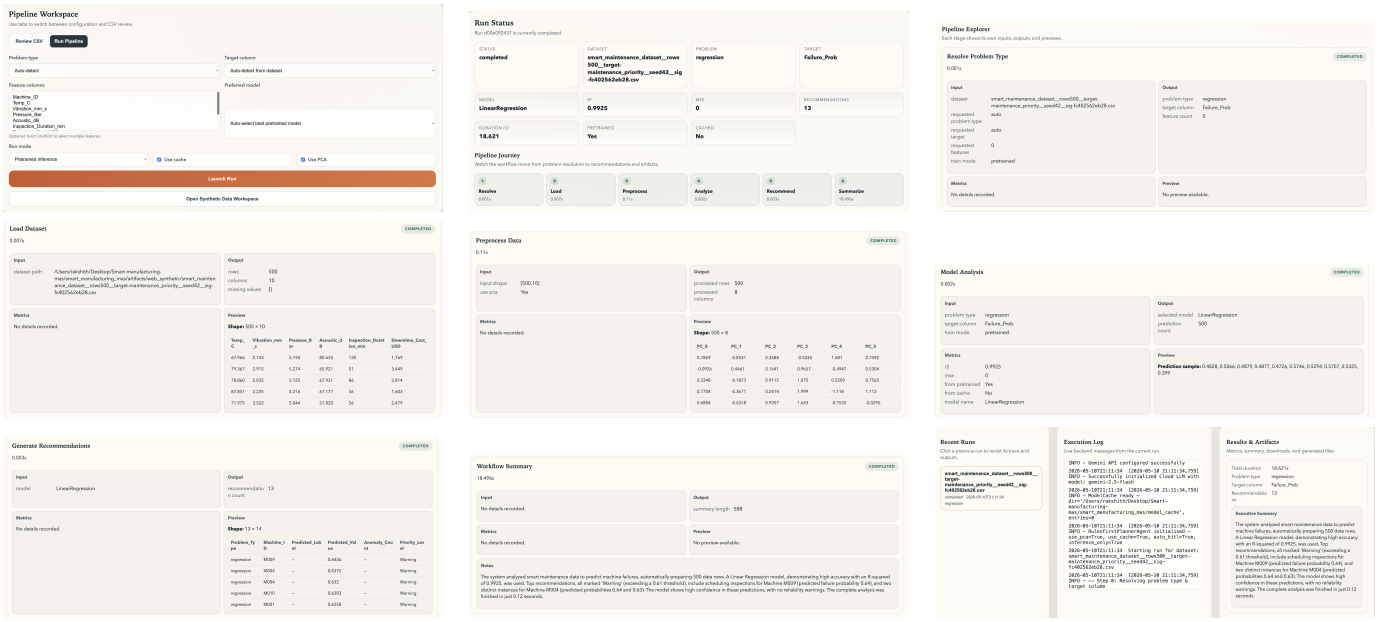

Fig. 2. Frontend outputs for predictions on synthetic data generated from the Smart Maintenance Dataset (500 rows).

it much easier to explain why a run behaved a certain way, and it also makes the system safer to extend.

There are still limits. If the input features are weakly related to the target, no amount of orchestration will create signal. Likewise, PCA can help with dimensionality but reduces interpretability, and anomaly detection still depends on assumptions about the rarity and structure of outliers. The repository handles these limits explicitly rather than hiding them.

IV. Conclusion

This project presents a practical smart-manufacturing workflow platform combining deterministic machine-learning pipelines with selective language-model orchestration. The implementation supports classification, regression, anomaly detection, synthetic-data evaluation, pretrained inference, frontend-assisted execution, caching, and Reflexion-based summarization within a unified repository.

A major contribution of the final implementation is the transition from a conceptual multi-SLM architecture toward a more reproducible and deployment-friendly rules-first workflow. Most language-model dependencies were replaced by deterministic logic, while one local SLM role was retained for anomaly-parameter estimation where flexible reasoning still provided value.

The repository therefore demonstrates how hybrid orchestration can be integrated with practical machine-learning infrastructure for smart-manufacturing applications. The result is not just a narrative architecture, but a working pipeline with visible stages, persisted artifacts, and repeatable execution modes.

V. Future Work

Future work will focus on transforming the current workflow into a fully closed-loop learning system. One proposed direction is an LLM-based reward mechanism that evaluates whether generated maintenance alerts were genuinely critical and uses that feedback to suppress false alerts or reinforce useful predictions.

Another important extension is live online training so that the system continuously adapts to incoming manufacturing data and retrains itself whenever predictions prove accurate. Additional improvements may include:

- online streaming sensor ingestion,
- continual learning,
- automatic drift detection,
- deployment on resource-constrained edge factory nodes,
- and tighter integration between synthetic-data generation and live production monitoring.

These additions would move the project closer toward fully autonomous smart-manufacturing decision support.

References

[1] M. A. Farahani, M. I. Khan, and T. Wuest, “Hybrid Agentic AI and Multi-Agent Systems in Smart Manufacturing,” preprint submitted to NAMRC / Manufacturing Letters, 2026.

[2] Rakhith, “Smart-manufacturing-mas,” GitHub repository, available: https://github.com/Rakhith/Smart-manufacturing-mas.